

Sorting Algorithm

Sorting Algorithm	Type	Best Time	Average Time	Worst Time	Space	Stable	In-Place
Bubble Sort	Comparison	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓
Selection Sort	Comparison	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗	✓
Insertion Sort	Comparison	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓
Merge Sort	Comparison	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓	✗
Quick Sort	Comparison	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	✗	✓
Heap Sort	Comparison	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗	✓
Counting Sort	Non-comparison	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	✓**	✗
Radix Sort	Non-comparison	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$	✓**	✗
Bucket Sort	Non-comparison	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Depends	✗
Tim Sort	Comparison	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓	✗

Important notes

- n = number of elements
- k = range/number of buckets depending on the algorithm
- d = number of digits/passes in Radix Sort

The ** was intended as a footnote marker:

- ✓* → Stable **depending on the implementation**
- ✓** → Stable **when implemented using the standard stable version**
-

1. Bubble Sort → Adjacent swapping
2. Selection Sort → Select minimum
3. Insertion Sort → Insert into sorted portion
4. Merge Sort → Divide + Merge
5. Quick Sort → Pivot + Partition

- 6. Heap Sort → Max Heap + Extract
- 7. Counting Sort → Frequency counting
- 8. Radix Sort → Digit by digit
- 9. Bucket Sort → Distribute into buckets
- 10. Tim Sort → Insertion + Merge

1. Bubble Sort

```
public class BubbleSort {  
  
    static void bubbleSort(int[] arr) {  
        int n = arr.length;  
  
        for (int i = 0; i < n - 1; i++) {  
            boolean swapped = false;  
  
            for (int j = 0; j < n - i - 1; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    int temp = arr[j];  
                    arr[j] = arr[j + 1];  
                    arr[j + 1] = temp;  
  
                    swapped = true;  
                }  
            }  
  
            if (!swapped) {  
                break;  
            }  
        }  
    }  
  
    public static void main(String[] args) {  
  
        int[] arr = {5, 3, 8, 4, 2};  
  
        bubbleSort(arr);  
  
        for (int n : arr) {  
            System.out.print(n + " ");  
        }  
    }  
}
```

2. Selection Sort

```
public class SelectionSort {  
  
    static void selectionSort(int[] arr) {  
        int n = arr.length;  
  
        for (int i = 0; i < n - 1; i++) {  
  
            int minIndex = i;
```

```

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

public static void main(String[] args) {

    int[] arr = {5, 3, 8, 4, 2};

    selectionSort(arr);

    for (int n : arr) {
        System.out.print(n + " ");
    }
}

```

3. Insertion Sort

```

public class InsertionSort {

    static void insertionSort(int[] arr) {

        for (int i = 1; i < arr.length; i++) {

            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {

        int[] arr = {5, 3, 8, 4, 2};

        insertionSort(arr);

        for (int n : arr) {
            System.out.print(n + " ");
        }
    }
}

```

4. Merge Sort

```
public class MergeSort {

    static void mergeSort(int[] arr, int left, int right) {

        if (left >= right) {
            return;
        }

        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }

    static void merge(int[] arr, int left, int mid, int right) {

        int[] temp = new int[right - left + 1];

        int i = left;
        int j = mid + 1;
        int k = 0;

        while (i <= mid && j <= right) {

            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
            }
        }

        while (i <= mid) {
            temp[k++] = arr[i++];
        }

        while (j <= right) {
            temp[k++] = arr[j++];
        }

        for (int x = 0; x < temp.length; x++) {
            arr[left + x] = temp[x];
        }
    }

    public static void main(String[] args) {

        int[] arr = {5, 3, 8, 4, 2};

        mergeSort(arr, 0, arr.length - 1);

        for (int n : arr) {
            System.out.print(n + " ");
        }
    }
}
```

5. Quick Sort

```
public class QuickSort {

    static void quickSort(int[] arr, int low, int high) {

        if (low < high) {

            int pivotIndex = partition(arr, low, high);

            quickSort(arr, low, pivotIndex - 1);
            quickSort(arr, pivotIndex + 1, high);

        }

    }

    static int partition(int[] arr, int low, int high) {

        int pivot = arr[high];

        int i = low - 1;

        for (int j = low; j < high; j++) {

            if (arr[j] < pivot) {

                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;

            }

        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        return i + 1;

    }

    public static void main(String[] args) {

        int[] arr = {5, 3, 8, 4, 2};

        quickSort(arr, 0, arr.length - 1);

        for (int n : arr) {
            System.out.print(n + " ");
        }

    }

}
```

6. Heap Sort

```
public class HeapSort {

    static void heapSort(int[] arr) {
```

```

    int n = arr.length;

    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    // Extract elements
    for (int i = n - 1; i > 0; i--) {

        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

static void heapify(int[] arr, int n, int i) {

    int largest = i;

    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    if (largest != i) {

        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

public static void main(String[] args) {

    int[] arr = {5, 3, 8, 4, 2};

    heapSort(arr);

    for (int n : arr) {
        System.out.print(n + " ");
    }
}
}

```

7. Counting Sort

This version also handles **negative numbers**.

```

public class CountingSort {

    static void countingSort(int[] arr) {

        if (arr.length == 0) {
            return;
        }

        int min = arr[0];
        int max = arr[0];

        for (int n : arr) {
            min = Math.min(min, n);
            max = Math.max(max, n);
        }

        int[] count = new int[max - min + 1];

        for (int n : arr) {
            count[n - min]++;
        }

        int index = 0;

        for (int i = 0; i < count.length; i++) {

            while (count[i] > 0) {
                arr[index++] = i + min;
                count[i]--;
            }
        }
    }

    public static void main(String[] args) {

        int[] arr = {5, -2, 3, 5, 1, -2, 4};

        countingSort(arr);

        for (int n : arr) {
            System.out.print(n + " ");
        }
    }
}

```

8. Radix Sort

This standard implementation works with **non-negative integers**.

```

public class RadixSort {

    static void radixSort(int[] arr) {

        if (arr.length == 0) {
            return;
        }

        int max = arr[0];

```

```

        for (int n : arr) {
            max = Math.max(max, n);
        }

        for (int exp = 1; max / exp > 0; exp *= 10) {
            countingSort(arr, exp);
        }
    }

    static void countingSort(int[] arr, int exp) {

        int n = arr.length;
        int[] output = new int[n];
        int[] count = new int[10];

        for (int value : arr) {
            int digit = (value / exp) % 10;
            count[digit]++;
        }

        for (int i = 1; i < 10; i++) {
            count[i] += count[i - 1];
        }

        // Reverse traversal makes counting sort stable
        for (int i = n - 1; i >= 0; i--) {

            int digit = (arr[i] / exp) % 10;

            output[count[digit] - 1] = arr[i];

            count[digit]--;
        }

        for (int i = 0; i < n; i++) {
            arr[i] = output[i];
        }
    }

    public static void main(String[] args) {

        int[] arr = {170, 45, 75, 90, 802, 24, 2, 66};

        radixSort(arr);

        for (int n : arr) {
            System.out.print(n + " ");
        }
    }
}

```

9. Bucket Sort

Here we use `ArrayList<Integer>` buckets.

```

import java.util.ArrayList;
import java.util.Collections;

public class BucketSort {

```

```

static void bucketSort(int[] arr) {

    if (arr.length == 0) {
        return;
    }

    int min = arr[0];
    int max = arr[0];

    for (int n : arr) {
        min = Math.min(min, n);
        max = Math.max(max, n);
    }

    int bucketCount = arr.length;

    ArrayList<ArrayList<Integer>> buckets = new ArrayList<>();

    for (int i = 0; i < bucketCount; i++) {
        buckets.add(new ArrayList<>());
    }

    for (int n : arr) {

        int index = (int) ((long) (n - min)
            * bucketCount
            / ((long) max - min + 1));

        buckets.get(index).add(n);
    }

    int index = 0;

    for (ArrayList<Integer> bucket : buckets) {

        Collections.sort(bucket);

        for (int n : bucket) {
            arr[index++] = n;
        }
    }
}

public static void main(String[] args) {

    int[] arr = {42, 32, 33, 52, 37, 47, 51};

    bucketSort(arr);

    for (int n : arr) {
        System.out.print(n + " ");
    }
}

```

10. Tim Sort

Tim Sort is a **hybrid of Insertion Sort and Merge Sort**. Below is a simplified implementation for learning purposes.

```
public class TimSort {

    static final int RUN = 32;

    static void timSort(int[] arr) {

        int n = arr.length;

        // Sort individual runs using insertion sort
        for (int i = 0; i < n; i += RUN) {

            insertionSort(
                arr,
                i,
                Math.min(i + RUN - 1, n - 1)
            );
        }

        // Merge sorted runs
        for (int size = RUN; size < n; size *= 2) {

            for (int left = 0; left < n; left += 2 * size) {

                int mid = Math.min(left + size - 1, n - 1);
                int right = Math.min(left + 2 * size - 1, n - 1);

                if (mid < right) {
                    merge(arr, left, mid, right);
                }
            }
        }
    }

    static void insertionSort(int[] arr, int left, int right) {

        for (int i = left + 1; i <= right; i++) {

            int key = arr[i];
            int j = i - 1;

            while (j >= left && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
    }

    static void merge(int[] arr, int left, int mid, int right) {

        int[] temp = new int[right - left + 1];

        int i = left;
        int j = mid + 1;
        int k = 0;

        while (i <= mid && j <= right) {
```

```

        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= right) {
        temp[k++] = arr[j++];
    }

    for (int x = 0; x < temp.length; x++) {
        arr[left + x] = temp[x];
    }
}

public static void main(String[] args) {

    int[] arr = {
        5, 3, 8, 4, 2,
        9, 1, 7, 6, 10
    };

    timSort(arr);

    for (int n : arr) {
        System.out.print(n + " ");
    }
}

```